Rheinisch-Westfälische Technische Hochschule Aachen
Alessandro Berti

User Guide For "Conformance Predictions"

Group 4:
Richard Schuppener, Cem Colak

Contents

1 Introduction **3**

2 Installation Guide **3**

2.1 Excerpt: Installation Guide from Requirements Engineering Document 3

3 User Guide **4**

3.1 Step 1: Event Log and Normative Model 4

3.1.1 Has model? 4

3.1.2 Event log path 4

3.1.3 Option A: Loading a Pre-Built Model 4

3.1.4 Option B: Discovering a Model 5

3.2 Step 2: Normative Model Visualization (Optional) 5

3.3 Step 3: Prediction Target 6

3.4 Step 4: Feature Type 7

3.5 Step 4.1: Prefix length 8

3.6 Step 5: ML Algorithm and Hyperparameters 8

3.7 Step 6: Decision Tree Visualization and Refinement 8

3.8 Step 7: Evaluation Results 9

3.9 Step 8: Saving Evaluation Visuals 10

3.10 Step 9: Predictions on New Data 10

3.11 Known Limitations of Sprint 1 10

1 Introduction

In this document we will provide a user guide for the "Conformance Predictions" project. The user guide is split into several sections. First, we will provide a guide for the installation of our project, followed by a guide for the usage of our software artifact.

2 Installation Guide

In our past Requirements Engineering document we have already outlined the Installation procedure for our project. In the following section we will provide the guide for completeness.

2.1 Excerpt: Installation Guide from Requirements Engineering Document

To use our **Python script**, the following set-up steps are required. First, the user should check whether all prerequisites are met: Our project requires a working **Python** installation of version *3.14.4* (currently the latest available Python version where all required packages are supported). Also, a proper installation of **Git** is required.

First, the repository needs to be cloned:

```
git clone
https://git.rwth-aachen.de/schuppenerr/conformance-predictions.git
```

Then, we suggest entering the cloned directory and creating a virtual environment for the Python packages:

```
python -m venv .venv
```

Alternatively (not recommended), this step can be skipped, and the required packages can be globally installed on the PC.

The next step is to install all packages from the *requirements.txt* file:

```
pip install -r requirements.txt
```

Now, the script can be launched via the terminal from the directory containing *main.py*:

```
python main.py
```

Note that this aims only to roughly sketch the installation process and neither depicts correct BPMN nor comprehensively describes the script setup.

3 User Guide

3.1 Step 1: Event Log and Normative Model

3.1.1 Has model?

`Do you have a normative process model? (y/n):`

Answer `y` if you have a pre-built model file ready, or `n` to let the tool discover a model automatically from the event log.

3.1.2 Event log path

`Enter the path to the event log (XES/CSV):`

Enter the absolute or relative path to your event log. Two file formats are accepted:

- **XES** (`.xes`) — the standard process mining log format; loaded directly via PM4Py.
- **CSV** (`.csv`) — the file must contain columns with exactly the names `case:concept:name`, `concept:name`, and `time:timestamp`. Any other column naming will cause a loading error.

3.1.3 Option A: Loading a Pre-Built Model

If you answered `y` in the first prompt:

`Enter the path to the normative model (Petri net/BPMN):`

Two formats are accepted:

- **Petri net** (`.pnml`) — loaded directly.
- **BPMN** (`.bpmn`) — converted to a Petri net internally before use.

Any other file extension will cause an error.

3.1.4 Option B: Discovering a Model

If you answered `n`, the tool prints:

```
Model discovery will be used.

Which discovery algorithm should be used?
1. Inductive Miner (recommended)
2. Alpha Miner
3. Genetic Miner
Select option (1-3):
```

Option 1 — Inductive Miner is the recommended choice for most logs. Two additional parameters are requested:

```
Noise threshold (default 0.0):
Disable fall-throughs? (y/n, default n):
```

The noise threshold (a value between 0.0 and 1.0) controls how aggressively infrequent behavior is filtered out before model construction. A value of 0.0 keeps all behavior; higher values produce simpler models. Press `Enter` to accept the defaults.

Option 2 — Alpha Miner has no configurable parameters. The tool will confirm this and continue:

```
Alpha Miner has no configurable parameters.
```

Option 3 — Genetic Miner requests six parameters:

```
Population size (default 500):
Elitism rate (default 0.01):
Crossover rate (default 1.0):
Mutation rate (default 0.01):
Generations (default 100):
Elitism min sample (default 5):
```

Press `Enter` on any field to keep its default value.

3.2 Step 2: Normative Model Visualization (Optional)

After the model is loaded or discovered, the tool asks:

```
Do you want a visualization of the [discovered/provided]
normative model saved (s) or directly displayed (d) or not (N)?
(s/d/N):
```

We recommend you to visualize the model at least once in the beginning to get a better understanding of the process structure. Especially for the prediction outcome target (option 2 in Step 3), it is important to know which activities signify the different outcomes. The visualization can be skipped in later runs once you are familiar with the model.

- **s — Save:** the model is rendered and written to a file. Two further prompts follow. First, choose the representation format:

  ```
  Choose model representation for visualization:
  1. Petri Net
  2. BPMN
  3. Process Tree
  Select option (1-3):
  ```

  Then provide the output file path:

  ```
  Enter the path to save the visualization (e.g., model.png):
  ```

- **d — Display:** the same representation prompt is shown, then the figure(png) is opened in an interactive window. No file is written.
- **N — Skip:** no visualization is produced, and the pipeline continues immediately.

3.3 Step 3: Prediction Target

```
What should be predicted?
1. Future conformance deviations
2. Process outcome
3. Remaining time
Select option (1-3):
```

Option 1 — Future conformance deviations predicts whether a case will exhibit at least one further deviation from the normative model. The label is binary: 0 means no further deviations are expected, 1 means at least one is expected. *Not implemented yet.*

Option 2 — Process outcome predicts which of several predefined outcomes a case will reach. After selecting this option, the following prompt appears:

```
Define the target outcome:
Enter activity names that signify a outcome (one per line).
Type 'done' when finished.
>
```

Enter each outcome activity name **exactly as it appears in the event log**, including capitalization, one name per line. Type `done` and press `Enter` when finished. Each case is labelled with the outcome activity that appears in it, provided exactly one of the specified activities is present. Cases containing none or more than one of the specified activities receive the label `uncertain_outcome`. These uncertain cases indicate, that the chosen outcome activities are not given. *Fully implemented.*

Option 3 — Remaining time. Not implemented in Sprint 1. The option is visible in the menu but selecting it will cause the pipeline to fail with an error.

3.4 Step 4: Feature Type

```
What should the prediction be based upon?
1. Plain trace (control-flow)
2. Trace and alignment-based diagnostics
3. Case features
4. Mixture of all available information
Select option (1-4):
```

Option 1 — Control-flow uses a bag-of-activities representation. For each case the feature vector contains the count of each activity observed in the log. *Fully implemented.*

Option 2 — Alignment-based diagnostics runs optimal alignment between the log and the model and encodes, for each case, the counts of synchronous moves, log-only moves, and model-only moves per activity. This is computationally more expensive than option 1 for large logs. *Fully implemented.*

Option 3 — Case features. Not implemented in Sprint 1. This option is intended to use static case-level attributes (e.g. a patient's age or admission department) stored in the event log. Selecting it will cause the pipeline to fail. It will be addressed in a future sprint.

Option 4 — Hybrid combines the features from options 1 and 2. Case attributes (option 3) are not yet included in the hybrid. *Partially implemented.*

3.5 Step 4.1: Prefix length

```
Enter the prefix length for prediction (e.g., 6). Leave empty for full traces:
```

For control-flow and alignment-based predictions of the process outcome, a prefix must be extracted from each trace. With this option, the user can specify a prefix length.

3.6 Step 5: ML Algorithm and Hyperparameters

```
Which ML algorithm do you want to apply on your data?
(1) Decision Tree
(2) Algorithm 2
(3) Algorithm 3
Select option (1-3):
```

Option 1 — Decision Tree trains a `Decision tree Classifier` from scikit-learn. It supports binary and multi-class prediction targets. *Fully implemented.*

Options 2 and 3 are placeholders. Selecting either will print a confirmation but will internally fall back to a Decision Tree with default parameters. The dedicated visualization and reconfiguration loop (Step 7) will also be skipped. These algorithms will be implemented in future sprints.

After selecting an algorithm:

```
Do you want to configure the parameters yourself? (y/n):
```

If you answer `y` and have selected the Decision Tree (option 1), four parameters can be set:

```
Criterion (gini/entropy, default gini):
Max depth (int, default 4):
Min samples split (int, default 20):
Min samples leaf (int, default 1):
```

Press `Enter` on any field to keep its default value. If you answer `n`, or if you selected option 2 or 3, the default parameters are applied automatically.

3.7 Step 6: Decision Tree Visualization and Refinement

This step only appears when the Decision Tree (option 1) is selected. We recommend, to visualize the trained tree at least once to understand how the

decision tree looks like with the default parameters and specified hyperparameters. After each visualization, you have the option to reconfigure the hyperparameters and retrain the model, such that you can interatively refine the tree structure and inspect the changes after each iteration.

After the initial training, the tool asks:

```
Do you want the decision tree saved (s),
displayed (d) or not shown (N)? (s/d/N):
```

- **s** — **Save:** provide a file path (e.g. `/Users/you/tree.png`). The tree is rendered with feature names and class labels and saved as an image file.
- **d** — **Display:** the tree is opened in an interactive Matplotlib window.
- **N** — **Skip:** no visualization is produced.

Immediately after the visualization choice, the tool asks:

```
Do you want to reconfigure the hyperparameters and retrain? (y/n):
```

If you answer `y`, the full Decision Tree parameter prompt from Step 6 is shown again. The model is retrained with the new settings and the visualization prompt is repeated, allowing you to inspect the new tree before deciding again. This loop continues until you answer `n`.

3.8 Step 7: Evaluation Results

Once the final model is confirmed, evaluation runs automatically. The terminal displays two outputs:

1. A **confusion matrix** printed as a text table, with rows representing actual class labels and columns representing predicted class labels.
2. A **per-class accuracy breakdown** listing, for each class, the percentage of cases that were correctly predicted and the percentage that were not.

Additionally, a summary line prints the scalar metrics: accuracy, precision, recall, F1-score, and ROC-AUC.

3.9 Step 8: Saving Evaluation Visuals

```
Save confusion matrix and accuracy chart? (y/n):
```

If you answer `y`:

```
Folder to save in (press Enter for Downloads):
```

Press `Enter` to save to your system's `Downloads` folder, or type a custom folder path. The tool creates a timestamped subfolder (e.g. `conformance_results_2026-05-15_14-30-00`) inside the chosen location and saves two image files:

- `confusion_matrix.png` — a color-coded heatmap of the confusion matrix.
- `per_class_accuracy.png` — a stacked bar chart with green segments (correct predictions) and red segments (incorrect predictions) per class.

3.10 Step 9: Predictions on New Data

```
Do you want to perform predictions on new data? (y/n):
```

Not implemented in Sprint 1. Answering `y` will prompt for a file path, but no predictions will be computed. The program exits after this step regardless of the answer.

3.11 Known Limitations of Sprint 1

The following features are visible in the CLI but not yet functional. Attempting to use them may cause the pipeline to produce incorrect results or terminate with an error.

- **Alpha Miner + Process Tree** The Alpha Miner does not guarantee to produce a Petri Net that is also a Workflow Net. The Process Tree visualization expects a workflow net (subclass of Petri Nets). This can result in an error.
- **Remaining time prediction** (target option 3) — not implemented; causes a pipeline error if selected.
- **Case attribute features** (feature option 3) — not implemented; causes a pipeline error if selected.

- **ML Algorithms 2 and 3** (options 2 and 3 in the algorithm menu) — placeholders only; the tool silently falls back to a Decision Tree with default parameters.
- **Hybrid features** (feature option 4) — functional for the combination of control-flow and diagnostics, but case attributes are not yet included.
- **Prediction on new data** (final step) — the prompt appears, but no prediction logic is executed.
- **CSV log format** — only supported if the file uses the exact column names `case:concept:name`, `concept:name`, and `time:timestamp`.
- **Multi-outcome cases** — if a case contains more than one of the specified outcome activities, it receives the label `uncertain_outcome` rather than a specific class.
- **Train/test split ratio** — the log is always split 75 % training / 25 % test using a temporal split by case end time. This ratio cannot be configured in Sprint 1.